Vault Enterprise Demo auf Hetzner Cloud

Geo-redundante Secrets-Management-Plattform auf Kubernetes (k3s) – Aufbau, Betrieb, Vorführung

Thomas Zachmann

Oktober 2026

Inhaltsverzeichnis

Überblick . . . 1
Architektur . . . 2
  Bausteine je Standort . . . 3
  Netzwerk und Entkopplung . . . 3
  Namen und TLS (nip.io + eigene Root-CA) . . . 4
  Auto-Unseal über den Unseal-Vault . . . 4
Kosten . . . 5
Voraussetzungen (einmalig) . . . 5
Bedienung . . . 6
  Ablauf von `mise run deploy` . . . 6
  Zugänge (nip.io) . . . 7
Drehbuch der Vorführung . . . 7
  1. Plattform zeigen . . . 8
  2. Hochverfügbarkeit im Cluster (Raft) . . . 8
  3. Anmeldung über das Behörden-IdM mit MFA . . . 8
  4. KV v2 – Versionierung . . . 8
  5. Kubernetes Auth und dynamische Datenbank-Zugänge . . . 8
  6. PKI – kurzlebige Zertifikate . . . 9
  7. Externe Anwendung: Zertifikats-Login und Transit . . . 9
  8. Audit und Monitoring . . . 9
  9. Disaster Recovery: Ausfall von RZ A . . . 10
  10. Ausfall des IdM (Break-Glass) . . . 10
Sicherheit: Root-Token und Notfallzugang . . . 10
Bekannte Vereinfachungen . . . 11
Ausbaustufen . . . 11
Teststand . . . 12
Fehlerbehebung . . . 13
Projektstruktur . . . 13
Referenzen . . . 13

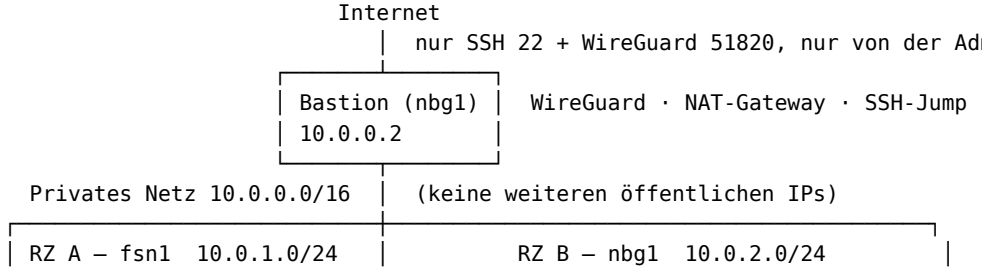
Überblick

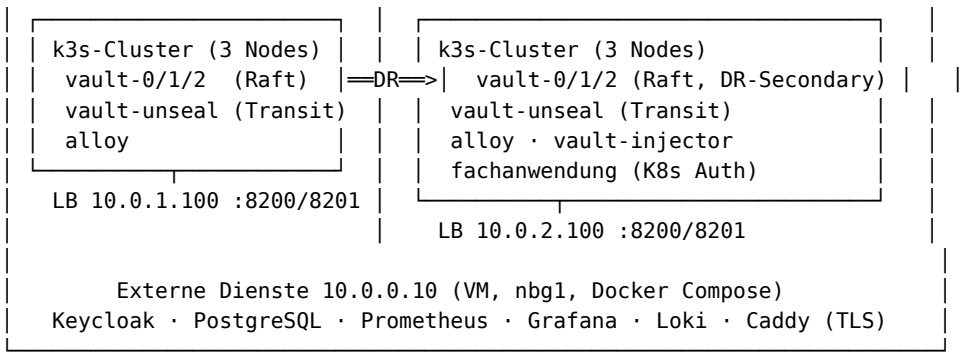
Diese Demo zeigt eine Referenzarchitektur für eine selbst betriebene, über zwei Rechenzentren verteilte Vault-Enterprise-Plattform einer öffentlichen Verwaltung – im Kleinen, auf der Hetzner Cloud in Deutschland. **Vault läuft in Kubernetes:** je Rechenzentrum ein k3s-Cluster, Installation über das offizielle Helm-Chart. Die Umgebung wird **nur für die Vorführung** aufgebaut

(`mise run deploy`, ca. 20 Minuten) und danach **vollständig gelöscht** (`mise run destroy`). Sie kostet ca. 0,34 € pro Stunde Laufzeit.

Referenzarchitektur	Demo
RZ A: 5 Vault-Nodes, Raft	**fsn1 (Falkenstein)**: k3s-Cluster mit 3 Nodes, Vault als Raft-StatefulSet mit 3 Pods (ein Pod pro Node)
RZ B: separater Cluster, DR-Replication	**nbg1 (Nürnberg)**: eigener k3s-Cluster, Vault als DR-Secondary von fsn1
Drei Failure Domains	Hetzner Spread-Placement-Group (physische Hosts) + Pod-Anti-Affinity
Interne Load Balancer mit TLS	Hetzner-LB je Standort, **nur privates Interface**, TCP-Passthrough auf den aktiven Vault-Pod
HSM-gestütztes Auto-Unseal	**Transit-Auto-Unseal** über einen separaten „Unseal-Vault“ je Standort (HSM-Stellvertreter)
Separate HSM-Infrastruktur je Standort	Eigener Unseal-Vault mit eigenem Schlüssel je Standort
Behördliches Identity Management	**Keycloak** (Realm `behoerde`), OIDC, TOTP-MFA verpflichtend
Zentrale Sicherheitsprotokollierung	Audit-Device → stdout → Grafana Alloy → **Loki**
Monitoring und Alerting	Alloy → **Prometheus** (Remote-Write) → **Grafana**
Namespaces je Fachverfahren	Vollständiger Namespace-Baum (Plattform, Fachverfahren, zentrale Dienste)
KV v2, Database, PKI, Transit	alle vier, mit Demo-Daten
Kubernetes Auth	**live**: Fachanwendung im Cluster, Vault Agent Injector, dynamische DB-Zugänge
OIDC mit MFA, Zertifikate, kurzlebige Tokens	live; GitLab-JWT nur beschrieben
Keine Root-Tokens im Betrieb	Alle Root-Tokens werden am Ende des Deployments **widerrufen**

Architektur





Warum Bastion und externe Dienste in nbg1? Die Demo simuliert den Ausfall von RZ A. Admin-Zugang, IdM und Fachdatenbank müssen das überstehen.

Warum externe Dienste auf einer VM statt im Cluster? Keycloak, Datenbank und Monitoring stehen in der Referenzarchitektur für **vorhandene Behörden-Infrastruktur** außerhalb der Vault-Plattform. Außerdem ist die Variante so genauso günstig wie eine reine VM-Lösung. Kämen die Dienste in den Cluster, bräuchte nbg1 größere Nodes (≈ +15 €/Monat).

Bausteine je Standort

Komponente	Umsetzung
Kubernetes	k3s `v1.36.5+k3s1`, 3 Server-Nodes mit embedded etcd (HA), Installation per cloud-init. Traefik und ServiceLB sind deaktiviert.
Vault	Helm-Chart `hashicorp/vault 0.34.1`, Image `vault-enterprise:2.1.2-ent` (ohne Lizenz `vault:2.1.2`), Raft mit `retry_join`, `service_registration "kubernetes"` (Label `vault-active`)
Storage	k3s `local-path` (je Pod ein Volume auf seinem Node; Raft repliziert die Daten selbst)
Unseal-Vault	eigenes Helm-Release im Namespace `vault-unseal`, 1 Pod (fest auf Node 3), Transit-Schlüssel `autounseal`
LB-Anbindung	Service `vault-lb` (NodePort 30200/30201) wählt nur den aktiven Pod; der Hetzner-LB zeigt auf diese NodePorts. Zusätzlich leitet der LB die Kubernetes-API (6443) weiter, damit kubectl jeden Node-Ausfall übersteht. Es läuft kein Cloud-Controller und kein Hetzner-Token im Cluster.
Monitoring-Agent	Alloy: scrapt die Vault-Pods, sammelt Pod-Logs und schreibt beides zur Dienste-VM

Netzwerk und Entkopplung

- **Nur die Bastion hat eine öffentliche IP.** Alle anderen Server sind ausschließlich im privaten Netz und gehen über die Bastion (NAT) ins Internet, z. B. für Images.

- **Admin-Zugriff** läuft per WireGuard (GUIs) bzw. SSH-Tunnel über die Bastion (kubectl, Terraform). Die Hetzner-Firewall lässt beides nur von der IP zu, von der aus deployt wurde.
- **Getrennte Cluster** je Standort mit eigener Kubernetes-API, eigenem etcd und eigenem Unseal-Vault. Ein Fehler oder Angriff im Cluster fsn1 erreicht den Cluster nbg1 nicht über Kubernetes.
- **Verbindungen zwischen den Standorten** laufen nur über definierte Pfade: Replikation über LB-Port 8201, Kubernetes-Auth-Prüfung von Vault zur API des Anwendungs-Clusters, IdM/DB/Monitoring zur Dienste-VM.

Vereinfachung: Ein Hetzner-Netz mit Subnetzen je Standort. Cloud-Firewalls filtern privaten Traffic nicht; in echten Rechenzentren trennen Firewalls die Standorte. Kubernetes-NetworkPolicies wären der nächste Härtungsschritt.

Namen und TLS (nip.io + eigene Root-CA)

- **nip.io** löst `<name>.<ip>.nip.io` auf `<ip>` auf, auch für private IPs. Dadurch gibt es lesbare Namen ohne eigene DNS-Zone, z. B. `vault-fsn1.10.0.1.100.nip.io`.
- **Let's Encrypt scheidet aus:** Die HTTP-Challenge braucht einen öffentlich erreichbaren Dienst, hier ist keiner öffentlich. Eine DNS-Challenge geht nicht, weil die nip.io-Zone nicht uns gehört.
- **Stattdessen eine Offline-Root-CA** (`.secrets/ca/root-ca.pem`). Sie steht für die übergeordnete behördliche PKI:
  - Sie wird einmal auf dem Laptop erzeugt und überlebt jedes `destroy`. Der Laptop muss ihr also nur **einmal** vertrauen.
  - Sie signiert die Server-Zertifikate (Vault, Unseal-Vault, Keycloak, Grafana) und die **Intermediate CA in Vault** (PKI-Engine). Vault übernimmt damit nur eine definierte Rolle innerhalb der PKI-Hierarchie.
- **Ausbaustufe:** Mit einer eigenen Domain (Subdomain an Hetzner DNS delegiert) wären Let's-Encrypt-Zertifikate per DNS-01 möglich, und `vault.demo.<domain>` ließe sich beim Failover per DNS umschalten.

DNS-Rebind-Schutz: Manche Router (z. B. FRITZ!Box) blockieren DNS-Antworten mit privaten IPs. Die WireGuard-Konfiguration setzt deshalb `DNS = 1.1.1.1`.

Auto-Unseal über den Unseal-Vault

Vault startet versiegelt. Damit Pods nach jedem Neustart ohne Menschen wieder arbeiten, entsiegeln sie sich über die **Transit-Engine eines zweiten, separaten Vault** im selben Standort:

1. `platform.sh` installiert den Unseal-Vault, initialisiert ihn (Shamir, 3 von 5) und legt den Transit-Schlüssel `autounseal` an. Außerdem erzeugt es ein Token, das **nur** `encrypt/decrypt` mit diesem Schlüssel darf (Kubernetes-Secret `vault-transit-token`).
2. Die drei Vault-Pods nutzen `seal "transit"`. Beim Init entstehen **Recovery-Keys** statt Unseal-Keys.
3. Fällt ein Vault-Pod aus oder startet ein Node neu, entsiegelt sich der Pod selbst.

Ehrlich benannt: - Der Unseal-Vault ist ein **HSM-Stellvertreter**. In der Referenzarchitektur steht hier ein HSM je Rechenzentrum (`seal "pkcs11"`). In Kubernetes ist SoftHSM nicht sinnvoll, weil es dafür kein offizielles Image mit HSM-Bibliothek gibt. - Der Unseal-Vault selbst wird per Shamir entsiegelt. Nach einem Neustart seines Nodes (Node 3) ist `mise run unseal` nötig. Ein echtes HSM hat dieses Problem nicht.

Kosten

Preise der Hetzner-Console, brutto inkl. 19 % USt., Stand Oktober 2026.

Komponente	Typ	Anzahl	€/Monat	€/Stunde
k3s-Nodes (je Standort 3)	CPX22 (2 vCPU, 4 GB)	6	139,14	0,222
Externe Dienste	CPX32 (4 vCPU, 8 GB)	1	42,23	0,068
Bastion	CPX12 (1 vCPU, 2 GB)	1	13,67	0,022
Load Balancer	LB11	2	17,82	0,029
Öffentliche IPv4	–	1	0,60	0,001
Netz, Firewalls, Traffic	–	–	0,00	–
Summe			**≈ 213**	**≈ 0,34**

Varianten im Vergleich: Vault direkt auf VMs kostet genauso viel (≈ 213 €). k3s mit allen Diensten im Cluster liegt bei ≈ 228 €, RKE2 bei ≈ 286 €, weil es mehr Grundlast mitbringt und größere Nodes braucht.

Nur bei Bedarf betreiben: Eine Vorführung (4 h) kostet ca. 1,40 €. Ausgeschaltete Server kosten weiter. Erst `mise run destroy` stoppt die Kosten, und das Skript prüft am Ende, dass im Projekt nichts übrig ist.

Voraussetzungen (einmalig)

1. **Werkzeuge:** `mise install` installiert die gepinnten Versionen von Terraform, Ansible, hcloud, kubectl, Helm, Vault-CLI, jq, Pandoc und Typst. Zusätzlich nötig: `envsubst` (`brew install gettext`) und Bash ≥ 4 (`brew install bash`).
2. **Hetzner-Projekt** „Vault“ mit API-Token (Lesen & Schreiben). Der Token liegt nur im macOS-Schlüsselbund. Die Eingabe gehört in ein **eigenes Terminal-Fenster**:

   ```
   security add-generic-password -U -a "$USER" -s hcloud-vault-demo -w
   ```

3. **SSH-Key** mit Passphrase, geladen im Agent:

   ```
   ssh-keygen -t ed25519 -f ~/.ssh/vault-demo -C vault-demo-admin
   ssh-add --apple-use-keychain ~/.ssh/vault-demo
   ```

4. **Root-CA erzeugen und dem Laptop vertrauen:**

   ```
   mise run ca:init
   sudo security add-trusted-cert -d -r trustRoot \
     -k /Library/Keychains/System.keychain "$PWD/.secrets/ca/root-ca.pem"   # im
   Projektverzeichnis ausführen
   ```

5. **WireGuard-App** aus dem Mac App Store.
6. **Vault-Enterprise-Lizenz** (30-Tage-Trial) unter `.secrets/vault.hclic` ablegen. Ohne Lizenz läuft die Demo im **Community-Modus**: gleiche Infrastruktur, Engines, Keycloak und Kubernetes Auth, aber ohne Namespaces (flache Pfade mit Präfix) und ohne DR-Replication.

Die Trial-Frist läuft ab Ausstellung. Erst anfordern, wenn der Community-Testlauf durchgelaufen ist.

`mise run setup` prüft alle Punkte.

Bedienung

Befehl	Wirkung
`mise run setup`	Voraussetzungen prüfen, Root-CA anlegen (falls nicht vorhanden)
`mise run deploy`	Komplettaufbau (idempotent, bei Fehlern einfach erneut starten)
`mise run urls`	GUI-Übersicht; `mise run urls -- --secrets` zeigt die Passwörter
`mise run status`	Vault-Pods, Raft-Peers und Replikationsstatus beider Standorte
`mise run kubectl -- nbg1 get pods -A`	kubectl gegen einen Cluster (über SSH-Tunnel)
`mise run ssh k3s-fsn1-1`	SSH auf einen Host (über die Bastion)
`mise run ha:node-fail` / `ha:node-restore`	Node 1 in fsn1 aus- und wieder einschalten
`mise run failover:simulate`	RZ A komplett ausschalten
`mise run failover:promote`	RZ B zum Primary befördern, Fachanwendung umstellen
`mise run failover:restore`	RZ A zurückholen, demoten, als DR-Secondary anhängen
`mise run unseal`	Unseal-Vaults nach Node-Neustart entsiegeln
`mise run destroy`	Alles löschen (fragt nach; `-- -y` ohne Rückfrage)
`mise run docs:pdf`	Dieses Dokument als PDF erzeugen

Ablauf von `mise run deploy`

1. **Terraform (`terraform/infra`):** SSH-Key, Netz und Subnetze, Bastion mit Firewall, 6 k3s-Nodes in zwei Placement-Groups, Dienste-VM, 2 private Load Balancer, Server-Zertifikate (signiert von der Root-CA), Ansible-Inventar.
2. **Bastion:** cloud-init richtet NAT und WireGuard ein. Das Skript holt die Client-Konfiguration nach `.secrets/wireguard/` und öffnet die SSH-Tunnel.
3. **Ansible (`ansible/services.yml`, parallel):** Docker-Stack der externen Dienste mit Keycloak-Realm, Datenbank mit Demo-Tabelle und Grafana-Dashboard.
4. **k3s:** Das Skript wartet auf je 3 Ready-Nodes und holt die kubeconfigs (`.secrets/kubeconfig-<site>`).
5. **`scripts/platform.sh` je Standort:** Secrets, Unseal-Vault (Init, Transit), Vault-Cluster (Helm, Init, Raft-Check), NodePort-Service und Service-Account für Kubernetes Auth.
6. **Alloy** in beiden Clustern.

7. **Terraform (`terraform/vault-config`):** Audit, Namespaces, Policies, OIDC/Keycloak, Break-Glass, KV-Daten, Database-, PKI- und Transit-Engine, Kubernetes Auth und Zertifikats-Auth.
8. **`app.sh`, `dr.sh`, `finalize.sh`:** Fachanwendung mit Injector, DR-Replication (nur Enterprise), Automatisierungs-Token, **Widerruf aller Root-Tokens**.

Zugänge (nip.io)

Alle Adressen sind nur mit **aktivem WireGuard-Tunnel** erreichbar (`.secrets/wireguard/vault-demo.conf` in die App importieren).

Dienst	URL
Vault RZ A (Primary)	`https://vault-fsn1.10.0.1.100.nip.io:8200/ui/`
Vault RZ B (DR)	`https://vault-nbg1.10.0.2.100.nip.io:8200/ui/`
Keycloak Admin	`https://keycloak.10.0.0.10.nip.io/admin/`
Keycloak Benutzerkonto	`https://keycloak.10.0.0.10.nip.io/realms/behoerde/account/`
Grafana	`https://grafana.10.0.0.10.nip.io/`
Prometheus	`https://prometheus.10.0.0.10.nip.io/`

Benutzer	Keycloak-Gruppe	Rechte in Vault
`alice`	`vault-admins`	Plattform-Administration (alle Namespaces)
`bob`	`fa-entwickler`	`fachverfahren-a/development`: KV lesen/schreiben
`carol`	`fb-betrieb`	`fachverfahren-b/production`: KV lesen/schreiben
`breakglass`	– (Userpass)	Notfallzugang, Token max. 1 h

Passwörter erzeugt jedes Deployment neu (`.secrets/passwords/`). Beim ersten Login richtet jeder Keycloak-Benutzer **TOTP** ein.

Drehbuch der Vorführung

Die CLI-Beispiele setzen voraus:

```
export VAULT_ADDR=https://vault-fsn1.10.0.1.100.nip.io:8200
export VAULT_CACERT=$PWD/.secrets/ca/root-ca.pem
```

Im **Community-Modus** entfällt `-namespace=…`. Die Pfade heißen dann z. B. `fachverfahren-a-development-kv/` statt `kv/` im Namespace. `terraform -chdir=terraform/vault-config output paths` zeigt die tatsächlichen Pfade.

1. Plattform zeigen

- `mise run kubectl -- fsn1 get pods -n vault -L vault-active`: drei Vault-Pods auf drei Nodes, genau einer mit `vault-active=true`
- `mise run status`: Raft-Peers, Seal-Typ `transit`, Replikationsstatus
- Grafana → Dashboard **Vault Demo**: beide Standorte, aktive Pods, Requests, Audit-Log

2. Hochverfügbarkeit im Cluster (Raft)

1. `mise run ha:node-fail`: Der Node mit dem **aktiven** Vault-Pod wird hart ausgeschaltet.
2. Raft wählt in Sekunden einen neuen Leader. **UI und API über den LB funktionieren weiter**, ebenso kubectl (etcd-Quorum 2 von 3, API über den LB).
3. `mise run ha:node-restore`: Der Node kommt zurück, der Pod entsiegelt sich über den Unseal-Vault selbst und wird Follower.

> **Beobachtung aus dem Test:** In den ersten ca. 40 Sekunden nach dem Ausfall können einzelne Anfragen fehlschlagen. Der tote Pod trägt noch das Label `vault-active=true`, denn sein Node kann es nicht mehr ändern. Kubernetes nimmt ihn erst aus dem Service, wenn der Node als NotReady gilt. Bei Vault auf VMs entfällt dieser Effekt, dort prüft der LB direkt `/sys/health`. Clients sollten deshalb Retries haben, was Vault Agent und die SDKs standardmäßig tun.

Botschaft: 3 Nodes vertragen 1 Ausfall, 5 Nodes (Referenzarchitektur) vertragen 2. Das Quorum ist $\lfloor n/2 \rfloor + 1$.

3. Anmeldung über das Behörden-IdM mit MFA

1. Vault-UI → Methode **OIDC** → „Sign in with OIDC Provider“.
2. Keycloak-Login als `bob`, TOTP einrichten bzw. eingeben.
3. Bob sieht nur `fachverfahren-a/development`. In `fachverfahren-a/production` erhält er *permission denied*, `fachverfahren-b` existiert für ihn nicht.
4. Abmelden und als `carol` anmelden: Sie sieht nur `fachverfahren-b/production`.

Botschaft: Vault verwaltet keine Personen. Gruppen kommen aus dem IdM und werden pro Namespace auf Policies gemappt. MFA erzwingt das IdM.

4. KV v2 – Versionierung

```
vault login -method=oidc       # als bob
vault kv get    -namespace=fachverfahren-a/development -mount=kv app-config
vault kv patch  -namespace=fachverfahren-a/development -mount=kv app-config
log_level=info
vault kv get    -namespace=fachverfahren-a/development -mount=kv -version=1 app-config
vault kv rollback -namespace=fachverfahren-a/development -mount=kv -version=1 app-
config
```

5. Kubernetes Auth und dynamische Datenbank-Zugänge

```
mise run kubectl -- nbg1 -n fachanwendung logs deploy/fachanwendung -c app -f
```

- Die Fachanwendung meldet sich mit ihrem **Service-Account-Token** an. Vault prüft es über die TokenReview-API des nbg1-Clusters (Rolle `fachanwendung`, gebunden an Service-Account und Namespace).
- Der **Vault Agent Injector** rendert `/vault/secrets/db.env` mit einem **eigens erzeugten PostgreSQL-Benutzer** (TTL 5 Minuten) und erneuert ihn automatisch. Im Log ist zu sehen, wie der Benutzername wechselt.
- Kein Passwort im Image, in Git oder in einem Kubernetes-Secret.

- Manuell als `alice`: `vault read -namespace=zentrale-dienste/database database/creds/fachverfahren-lesen`
- **Root-Rotation:** `vault write -namespace=zentrale-dienste/database -f database/rotate-root/fachverfahren`. Danach kennt nur noch Vault das Passwort seines DB-Benutzers.

6. PKI – kurzlebige Zertifikate

```
vault write -namespace=zentrale-dienste/certificates pki/issue/interne-dienste \
  common_name=melderegister.svc.behoerde.demo ttl=24h
```

Die Kette ist: Zertifikat → **Vault Demo Issuing CA** (in Vault) → **Vault Demo Root CA** (offline, Laptop).

7. Externe Anwendung: Zertifikats-Login und Transit

```
vault write -format=json -namespace=zentrale-dienste/certificates \
  pki/issue/externe-anwendungen common_name=portal.apps.behoerde.demo ttl=1h > /tmp/
app.json
jq -r .data.certificate /tmp/app.json > /tmp/app.pem
jq -r .data.private_key /tmp/app.json > /tmp/app.key

vault login -namespace=platform/shared-services -method=cert \
  -client-cert=/tmp/app.pem -client-key=/tmp/app.key name=externe-anwendung

vault write -namespace=platform/shared-services transit/encrypt/fachdaten \
  plaintext=$(echo -n "Personalausweis L01X00T47" | base64)
```

Anschließend als `alice` den Schlüssel rotieren (`transit/keys/fachdaten/rotate`). Alte Chiffretexte bleiben entschlüsselbar, `transit/rewrap/fachdaten` hebt sie auf die neue Version.

8. Audit und Monitoring

- Grafana → **Vault Demo**: Das Audit-Log beider Standorte zeigt jeden Zugriff mit Benutzer, Namespace, Pfad und Operation. Secret-Werte sind HMAC-geschützt.
- **Break-Glass-Nutzung** sichtbar machen: Login mit Methode *Userpass*, Pfad `breakglass`. Im Audit-Log erscheint `breakglass-breakglass`. In Produktion wäre das ein Alarm.

9. Disaster Recovery: Ausfall von RZ A

Schritt	Befehl	Was man sieht
1. Ausgangslage	`mise run status`	fsn1 `mode=primary`, nbg1 `mode=secondary state=stream-wals`
2. Ausfall	`mise run failover:simulate`	Alle Nodes in fsn1 sind aus, die fsn1-UI ist weg. Die Fachanwendung bekommt keine neuen DB-Zugänge mehr.
3. Wiederanlauf	`mise run failover:promote`	nbg1 wird Primary mit **allen** Secrets, Policies, Namespaces und Leases. Der Injector zeigt auf nbg1, die Fachanwendung läuft weiter.
4. Prüfung	Login auf `vault-nbg1…`	Bob, Carol und alle Daten wie vorher
5. Rückkehr RZ A	`mise run failover:restore`	fsn1 wird demotet (kein Split-Brain) und hängt als neuer DR-Secondary an nbg1

Wichtige Details:

- **Befördert wird mit einem DR-Operation-Batch-Token.** Es darf ausschließlich `promote/update-primary`, wurde vorab erzeugt und funktioniert ohne den ausgefallenen Primary. Ein Root-Token ist nicht nötig.
- **Die Beförderung ist bewusst manuell.** Automatischer Failover zwischen Rechenzentren riskiert Split-Brain bei Netztrennung. Die Referenzarchitektur verlangt einen *definierten* Wiederanlaufprozess, und dieser Ablauf ist er.
- **Replikation über die LBs:** Pod-IPs sind zwischen den Clustern nicht routbar. Deshalb wird `primary_cluster_addr` beim Aktivieren bzw. Befördern auf den LB-Port 8201 gesetzt.
- **Kubernetes Auth funktioniert nach dem Failover weiter.** Die replizierte Konfiguration prüft Tokens gegen die API des Anwendungs-Clusters (nbg1), und der ist nicht ausgefallen.

10. Ausfall des IdM (Break-Glass)

1. Keycloak stoppen: `mise run ssh services -- docker stop services-keycloak-1`
2. Der OIDC-Login scheitert.
3. Login mit Methode *Userpass*, Pfad `breakglass` und Passwort aus `mise run urls -- --secrets`. Das Token gilt höchstens 1 Stunde.
4. Keycloak wieder starten: `docker start services-keycloak-1`.

Sicherheit: Root-Token und Notfallzugang

- Am Ende des Deployments werden **alle Root-Tokens widerrufen**: beide Vault-Cluster und beide Unseal-Vaults. Für Skripte gibt es ein Automatisierungs-Token mit 8 h fester Laufzeit (`.secrets/automation-token`).

- Ein neues Root-Token gibt es nur per `vault operator generate-root` mit Quorum der **Recovery-Keys** (3 von 5, `.secrets/init-fsn1.json`). **Neu ab Vault 2.0:** Der Endpunkt verlangt zusätzlich ein gültiges Token (z. B. Break-Glass), oder `enable_unauthenticated_access = ["generate-root"]` wird gesetzt.
- In Produktion werden Recovery-Keys auf 5 Personen verteilt (PGP-verschlüsselt per `-recovery-pgp-keys`) und nie gemeinsam gespeichert. In der Demo liegen sie zur Vereinfachung lokal in `.secrets/`.

Bekannte Vereinfachungen

Thema	Demo	Produktion
HSM	Unseal-Vault mit Transit	Netzwerk-HSM je RZ (`seal "pkcs11"`)
Failure Domains	Placement-Group (Hosts)	Brandabschnitte/Räume, Topology-Spread
Storage	`local-path`	Replizierter/performanter Block-Storage (CSI)
Standort-Trennung	Ein Hetzner-Netz	Getrennte Netze, Firewalls, NetworkPolicies
Recovery-Keys, Tokens	lokal in `.secrets/`	Auf Personen verteilt, PGP
Datenbank-Verbindung	ohne TLS	`sslmode=verify-full`
Externe Dienste	eine VM, Docker Compose	Behörden-Infrastruktur, hochverfügbar
Prometheus/Loki	ohne Authentifizierung, nur privat	mTLS bzw. Authentifizierung, Retention-Konzept
Kubernetes	k3s, Kubeconfig mit Admin-Rechten	RKE2/OpenShift, OIDC-Login, RBAC
Failover-Erkennung im Cluster	Node-NotReady nach ~40 s	Kürzere `node-monitor-grace-period`, Readiness auf `/sys/health`
SSH	`root` mit Key	Persönliche Konten, SSH-CA

Ausbaustufen

- **GitOps mit Argo CD:** Die Fachanwendung wird per Argo CD ausgerollt und bezieht ihre Secrets über den **Vault Secrets Operator**. Damit liegen nie Secrets in Git. Für den Vault-Bootstrap selbst (Init, Unseal, DR) bleibt es bei den Skripten, weil diese Schritte imperativ sind.
- **GitLab CI:** `jwt`-Auth mit `bound_claims` auf Projekt und Branch, Runner im privaten Netz.
- **Eigene Domain:** Let's Encrypt per DNS-01, DNS-basierter Failover-Name.
- **5 statt 3 Nodes je Standort:** `nodes_per_site = 5` in `terraform/infra/variables.tf`, `server.ha.replicas` und `retry_join` in `k8s/vault/values-vault.yaml` anpassen.

Teststand

Live auf Hetzner getestet (Community-Modus, 18.10.2026):

Bereich	Ergebnis
Infrastruktur, NAT, WireGuard-Konfiguration	✓
2 k3s-Cluster mit je 3 Nodes (embedded etcd)	✓
Vault je Standort: 3 Pods, Raft, Transit-Auto-Unseal	✓
Keycloak, PostgreSQL, Prometheus, Grafana, Loki (TLS über eigene Root-CA)	✓
OIDC-Discovery gegen Keycloak, Break-Glass-Login	✓
KV v2, dynamische DB-Zugänge, PKI (Kette bis zur Root-CA), Transit	✓
Zertifikats-Login einer externen Anwendung	✓
Kubernetes Auth + Agent Injector (Fachanwendung liest Tabelle mit wechselnden DB-Benutzern)	✓
Metriken beider Standorte in Prometheus, Audit-Log in Loki	✓
HA: Ausfall des Nodes mit aktivem Pod, Leaderwechsel, Rückkehr	✓
Root-Token-Widerruf, `destroy` mit Leer-Kontrolle	✓
Namespaces, DR-Replication, Failover	**offen – braucht die Enterprise-Lizenz**
Browser-Login mit TOTP über WireGuard	offen – nur interaktiv am Laptop prüfbar

Fehlerbehebung

Symptom	Ursache und Lösung
`terraform apply`: Servertyp nicht verfügbar	Hetzner meldet Typen zeitweise als ausverkauft. `server_type` in `terraform/infra/variables.tf` ändern.
Deploy hängt bei „k3s-Cluster … Ready“	NAT der Bastion prüfen: `mise run ssh bastion -- systemctl status nat-masquerade`, cloud-init-Log auf dem Node: `mise run ssh k3s-fsn1-1 -- tail /var/log/cloud-init-output.log`
Vault-Pods bleiben `0/1 Ready`	Unseal-Vault versiegelt: `mise run unseal`, Logs: `mise run kubectl -- fsn1 -n vault logs vault-0`
Browser: Zertifikatsfehler	Root-CA nicht vertraut (siehe Voraussetzungen), Browser neu starten
nip.io-Name löst nicht auf	DNS-Rebind-Schutz im Router, WireGuard nicht aktiv
SSH/Tunnel verweigert nach IP-Wechsel	Firewall erlaubt nur die Deploy-IP: `mise run deploy` erneut (idempotent)
Automatisierungs-Token abgelaufen (8 h)	Mit Break-Glass oder `generate-root` ein neues erzeugen, oder neu deployen

Projektstruktur

```
mise.toml                    Tools + Tasks (deploy, destroy, urls, failover …)
scripts/                     deploy, platform (Vault je Standort), app, dr, finalize,
                             destroy, status, ha, failover, unseal, urls, ssh, kubectl
terraform/infra/             Hetzner: Netz, Bastion, k3s-Nodes, Dienste-VM, LBs,
Zertifikate
terraform/vault-config/      Vault: Namespaces, Engines, Auth-Methoden, Policies
k8s/vault/                   Helm-Values (Vault, Unseal-Vault), NodePort-Service
k8s/monitoring/alloy.yaml    Metriken + Logs → Dienste-VM
k8s/fachanwendung/app.yaml   Demo-Workload mit Vault Agent Injector
ansible/                     Dienste-VM (Keycloak, PostgreSQL, Monitoring)
archive/k8s-entwurf/         Verworfener Entwurf "alles im Cluster" (teurer)
docs/vault-demo.md           Diese Dokumentation (PDF: mise run docs:pdf)
.secrets/                    Root-CA, Lizenz, Keys, Tokens, kubeconfigs (gitignored)
```

Referenzen

- Vault auf Kubernetes (Helm): https://developer.hashicorp.com/vault/docs/platform/k8s/helm
- Integrated Storage (Raft): https://developer.hashicorp.com/vault/docs/configuration/storage/raft
- Transit-Seal: https://developer.hashicorp.com/vault/docs/configuration/seal/transit
- DR-Replication: https://developer.hashicorp.com/vault/docs/enterprise/replication
- Kubernetes Auth: https://developer.hashicorp.com/vault/docs/auth/kubernetes
- Vault Agent Injector: https://developer.hashicorp.com/vault/docs/platform/k8s/injector
- Namespaces: https://developer.hashicorp.com/vault/docs/enterprise/namespaces

- Wichtige Änderungen in Vault 2.x: https://developer.hashicorp.com/vault/docs/updates/important-changes
- k3s: https://docs.k3s.io
- nip.io: https://nip.io